

Flutter, Flame & Fragment shaders

reNA



Renan Araujo

GDE, Bluefire, Superlist

@reNotANumber

NAN

@reNotANumber

Flutter

Flame

Fragment Shaders

Agenda

- Crash course on Fragment Shaders
- The different ways people use them with Flutter
- Quick Look at Flutter Post-processing
- Flame overview and introduction to Post Process

Fragment Shaders


```
#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;

out vec4 fragColor;

void main() {
    vec2 pos = FlutterFragCoord().xy;
    vec2 uv = pos / uSize;

    float red = uv.x;
    float green = uv.y;
    float blue = 1.0 - red;

    fragColor = vec4(red, green, blue, 1.0);
}
```



```
#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;

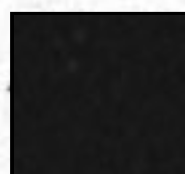
out vec4 fragColor;

void main() {
    vec2 pos = FlutterFragCoord().xy;
    vec2 uv = pos / uSize;

    float red = uv.x;
    float green = uv.y;
    float blue = 1.0 - red;

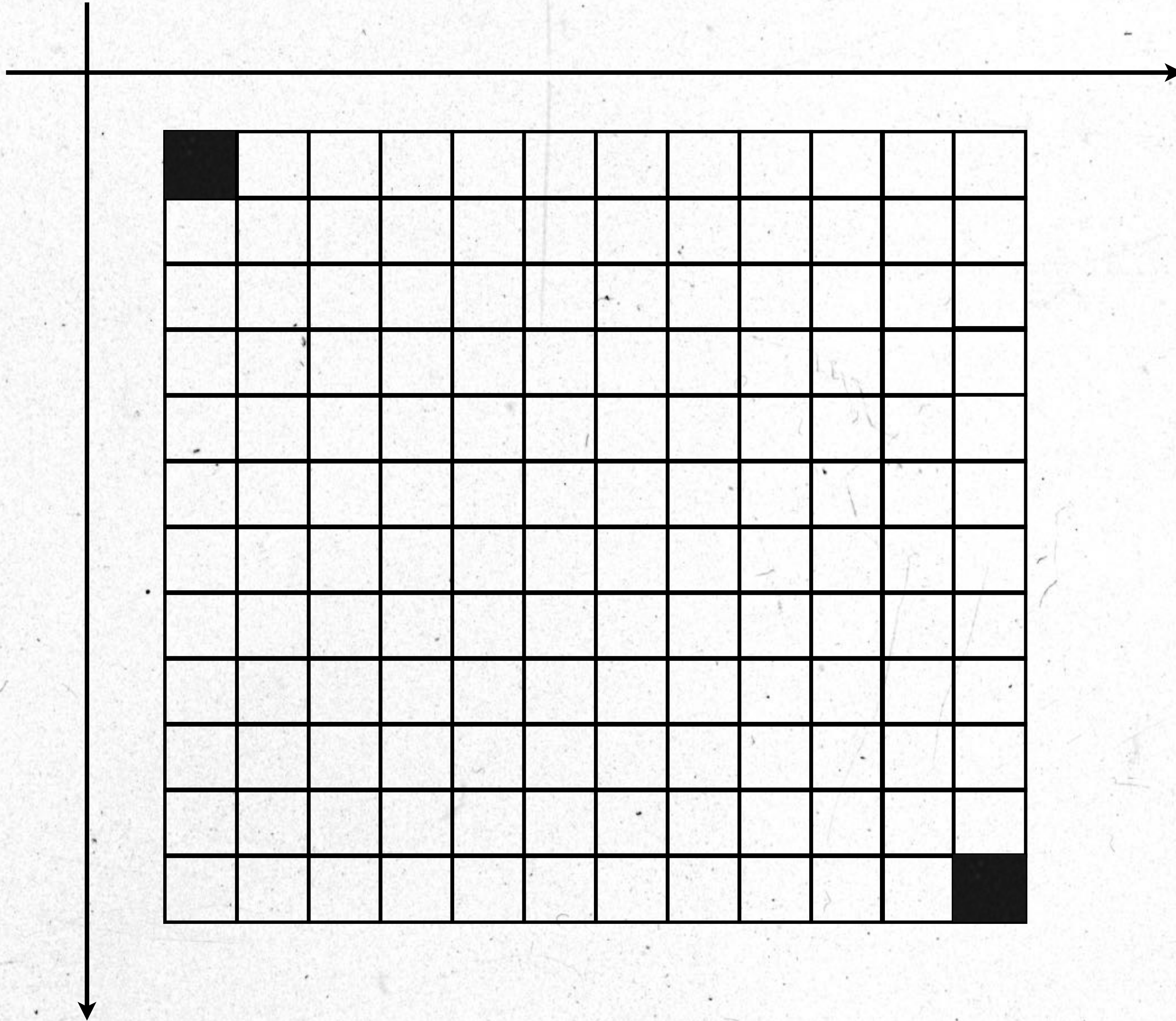
    fragColor = vec4(red, green, blue, 1.0);
}
```

What color should A pixel have?



NAN

@reNotANumber



NAN


```
vec2 uv = vec2(0.0, 0.0);
```

```
float red = uv.x;
```

```
float green = uv.y;
```

```
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN

@reNotANumber


```
vec2 uv = vec2(0.0, 0.0);
```

```
float red = 0.0;
```

```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN

@reNotANumber


```
vec2 uv = vec2(0.0, 0.0);
```

```
float red = 0.0;
```

```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN

@reNotANumber



```
vec2 uv = vec2(1.0, 1.0);
```

```
float red = uv.x;  
float green = uv.y;  
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



```
vec2 uv = vec2(1.0, 1.0);
```

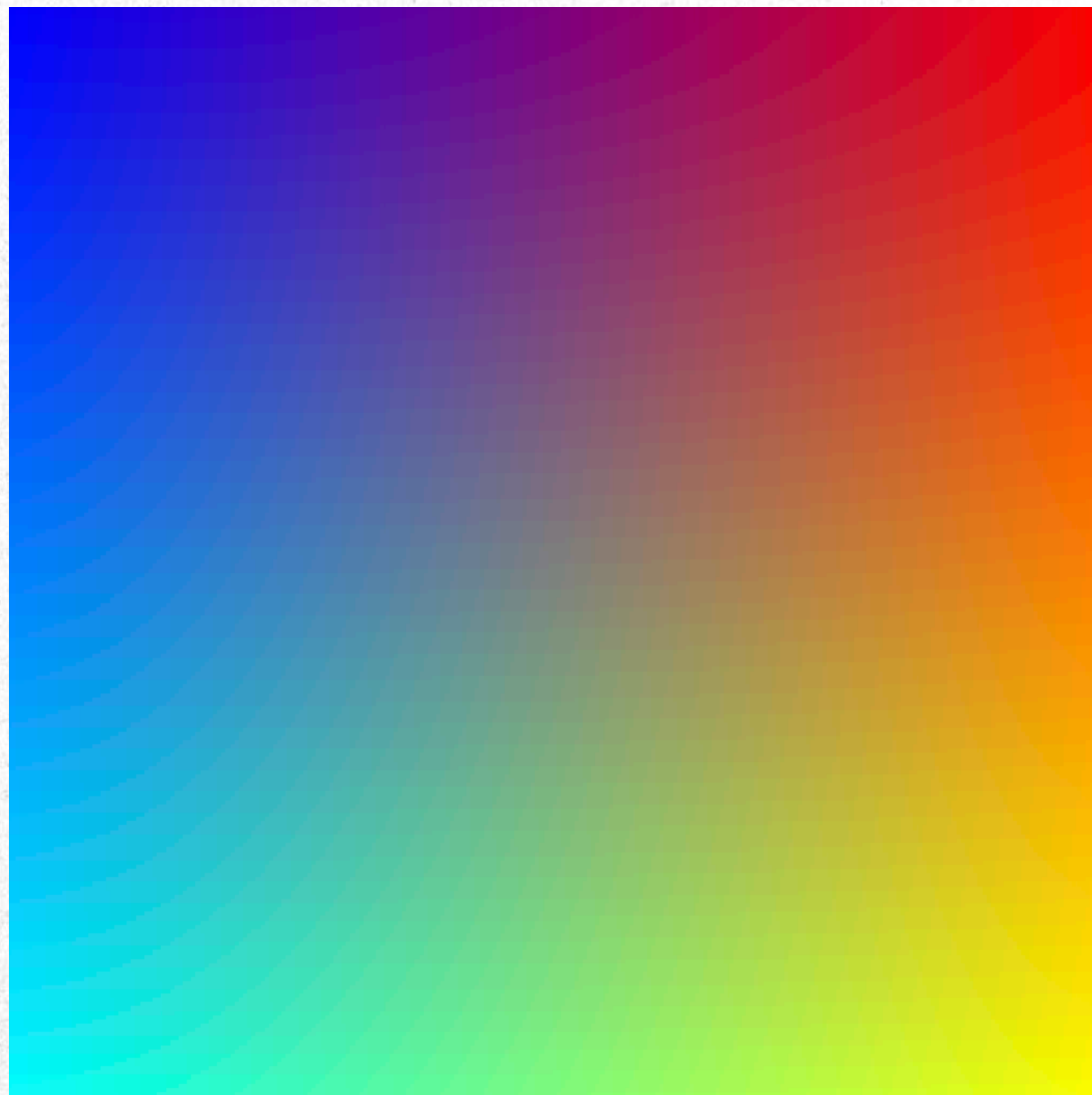
```
float red = 1.0;
```

```
float green = 1.0;
```

```
float blue = 1.0 - 1.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



NAN

@reNotANumber



```
uniform float uScale;  
uniform vec2 uSize;  
uniform vec4 uColor;  
uniform mat4 uTransform;  
uniform sampler2D uTexture;
```

Inputs

- A series of numeric types (float, vectors, matrices, arrays...)
- Textures

Flutter

Fragment Shaders

Flutter

Fragment Shaders

Possible inputs

- Scroll position
- Colors
- Gesture coordintes
- Sizes
- Animated values

Ways to render Shaders on Flutter

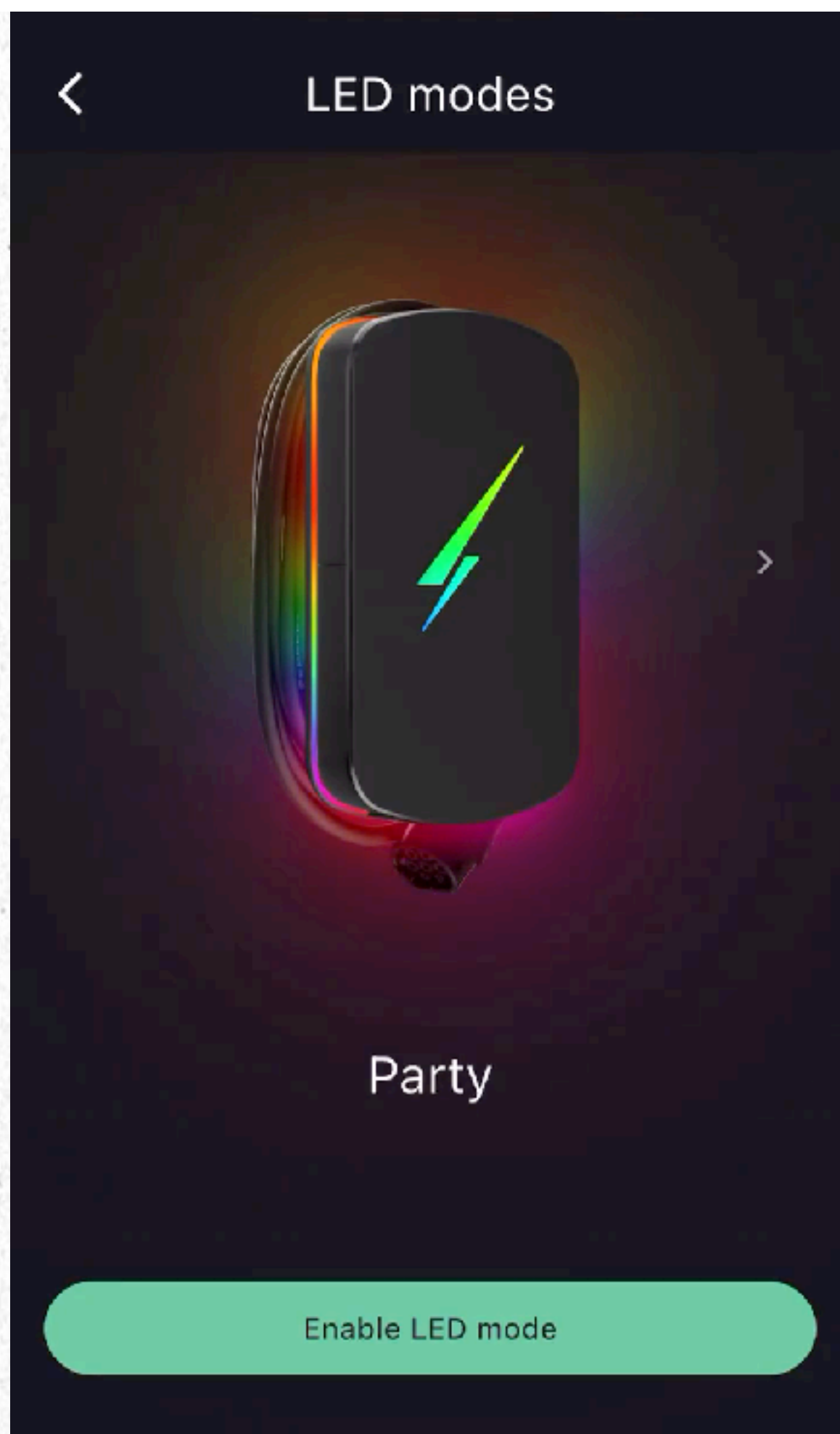
1. Just paint it


```
final paint = Paint()  
    ..shader = shader;  
  
shader.setFloatUniforms((s) {  
    s.setSize(size);  
});  
  
canvas.drawRect(Offset.zero & size, paint);
```

Canvas based APIs

CustomPainter, Render Objects





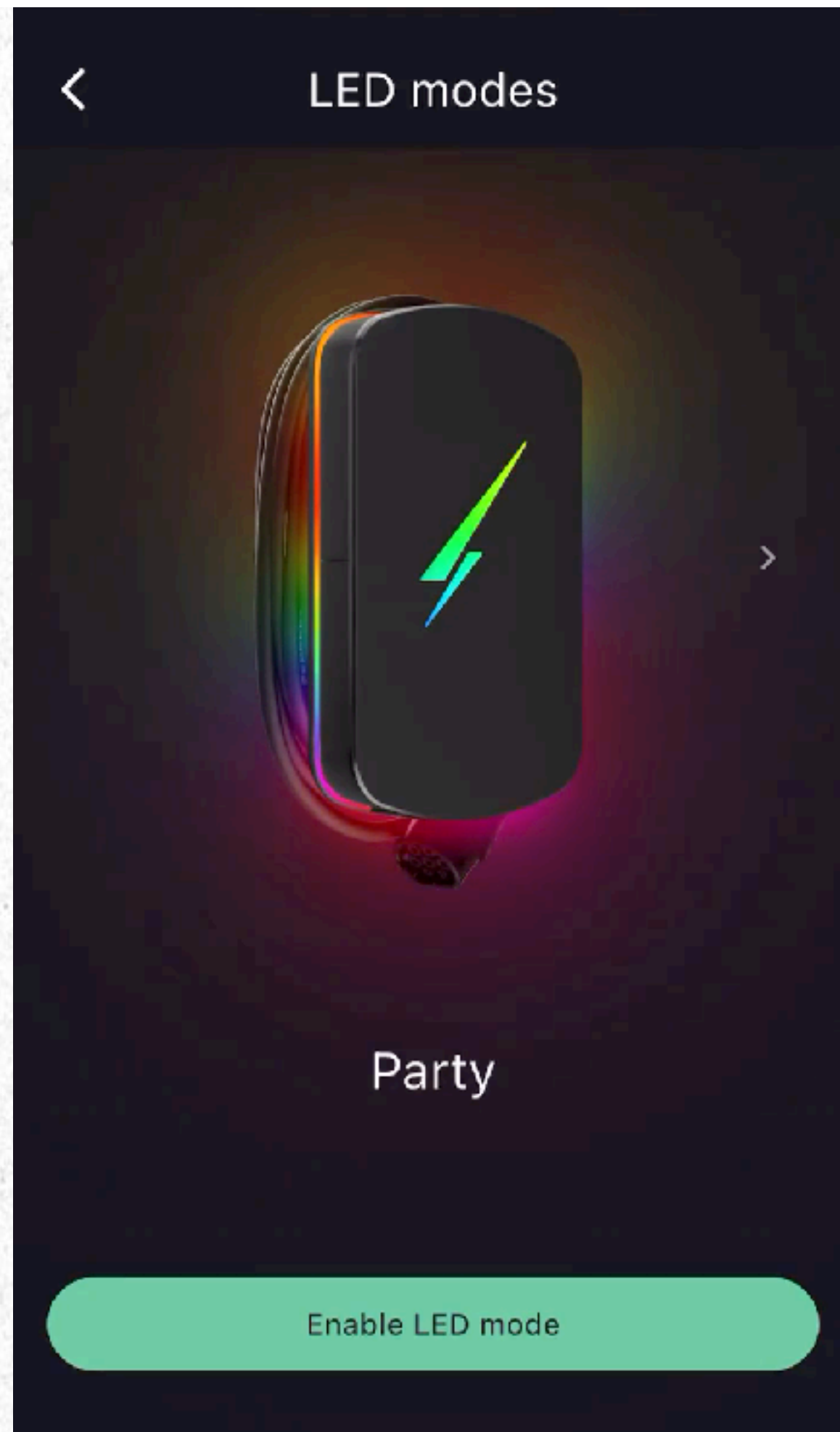
Hypervolt

Render light effects

hypervolt.co.uk

NAN

@reNotANumber



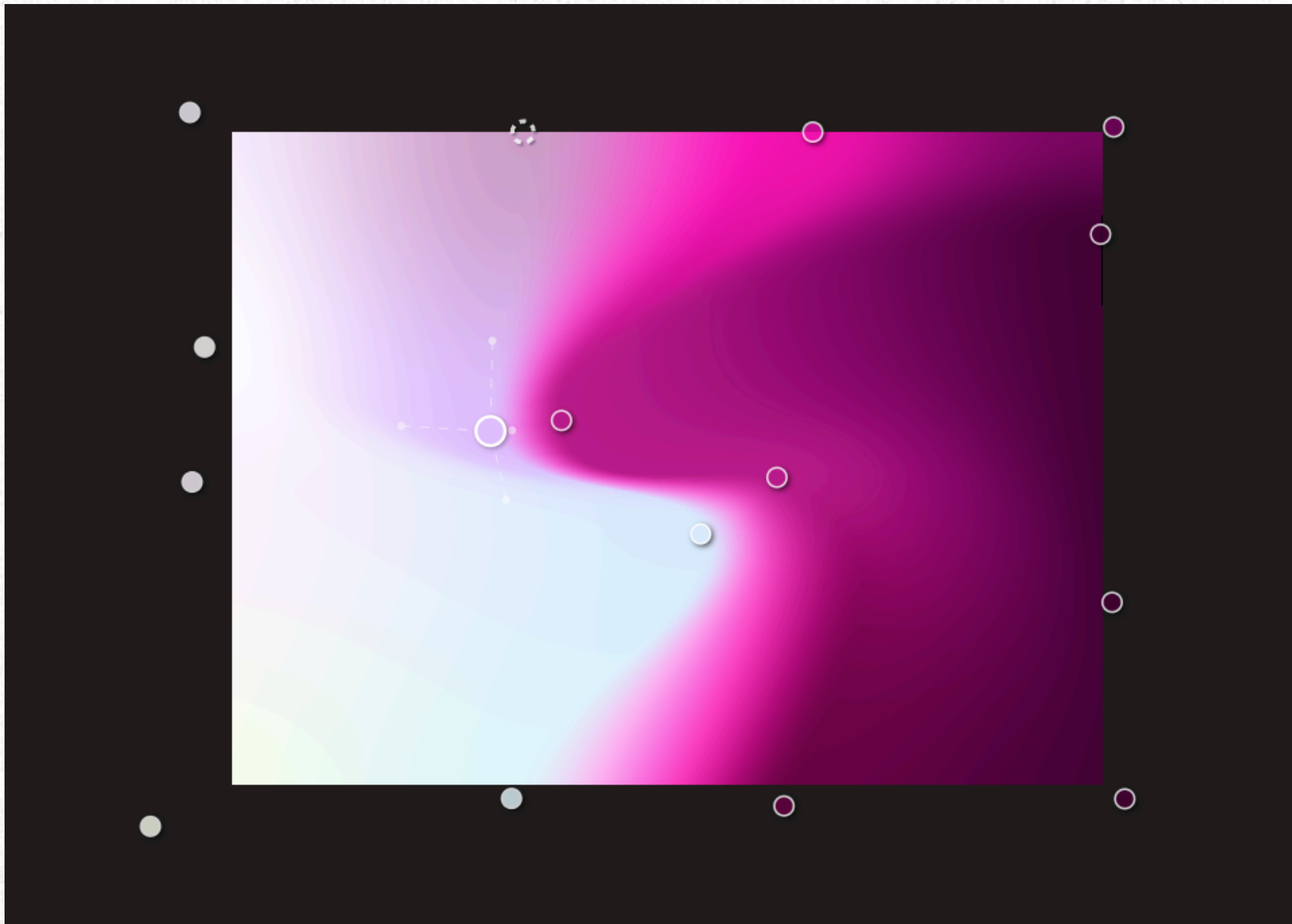
Hypervolt

Render light effects

hypervolt.co.uk

NAN

@reNotANumber



O'Mesh

Uses positions of control points and
the colors

omesh.renan.gg

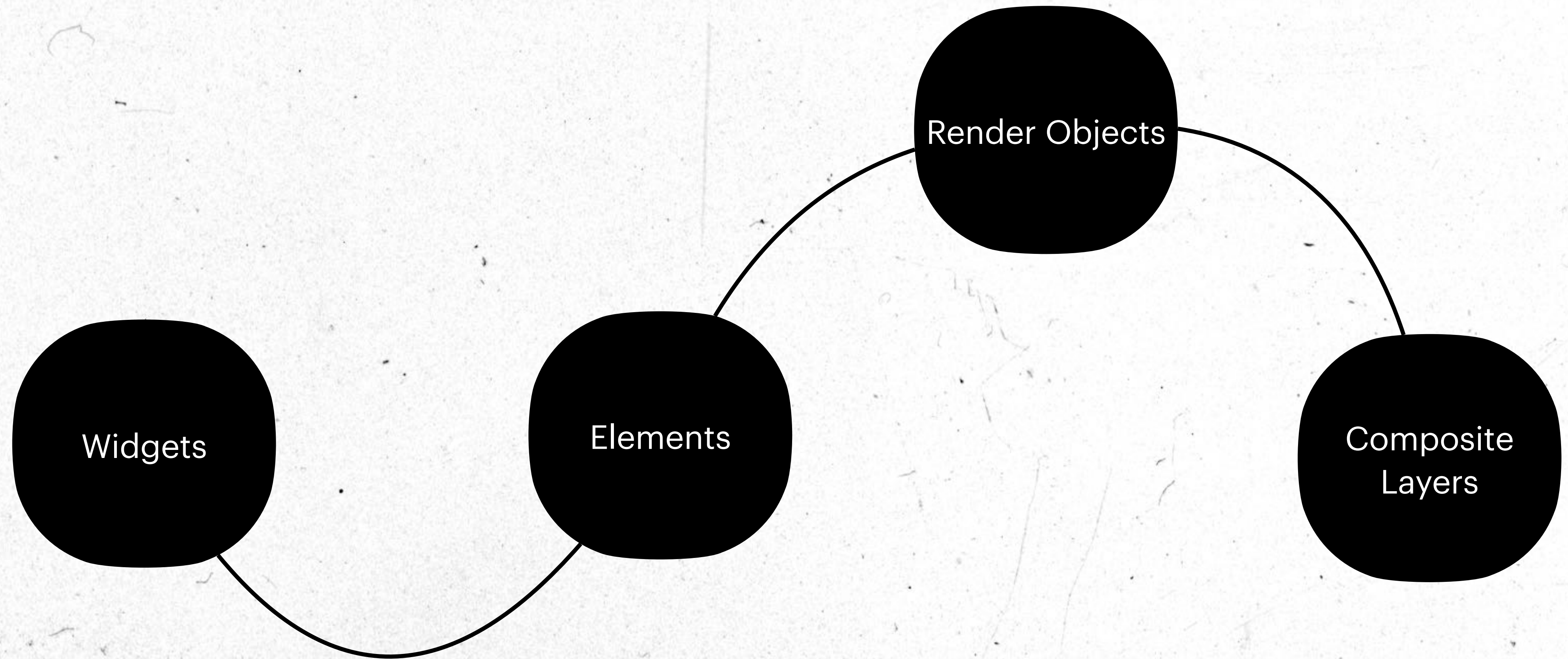
2. Sample children

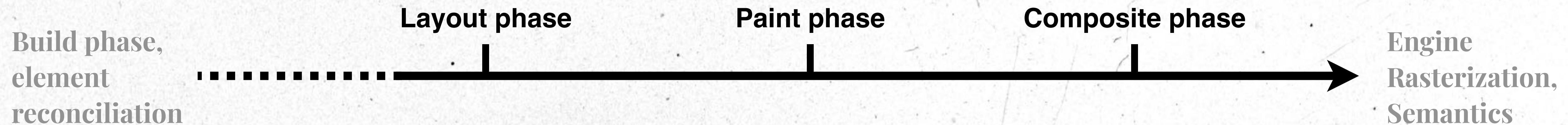
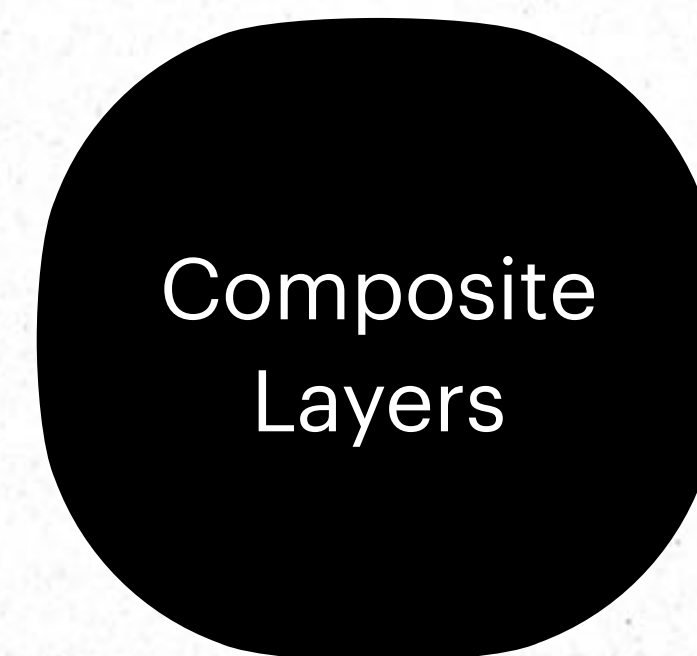

```
Widget build(BuildContext context) {  
  return AnimatedSampler(  
    (ui.Image image, Size size, Canvas canvas) {  
      shader  
        ..setFloat(0, size.width)  
        ..setFloat(1, size.height)  
        ..setImageSampler(0, image);  
      canvas.drawRect(  
        Offset.zero & size,  
        Paint()..shader = shader,  
      );  
    },  
    child: widget.child,  
  );  
}
```

Animated sampler

from the package flutter_shaders

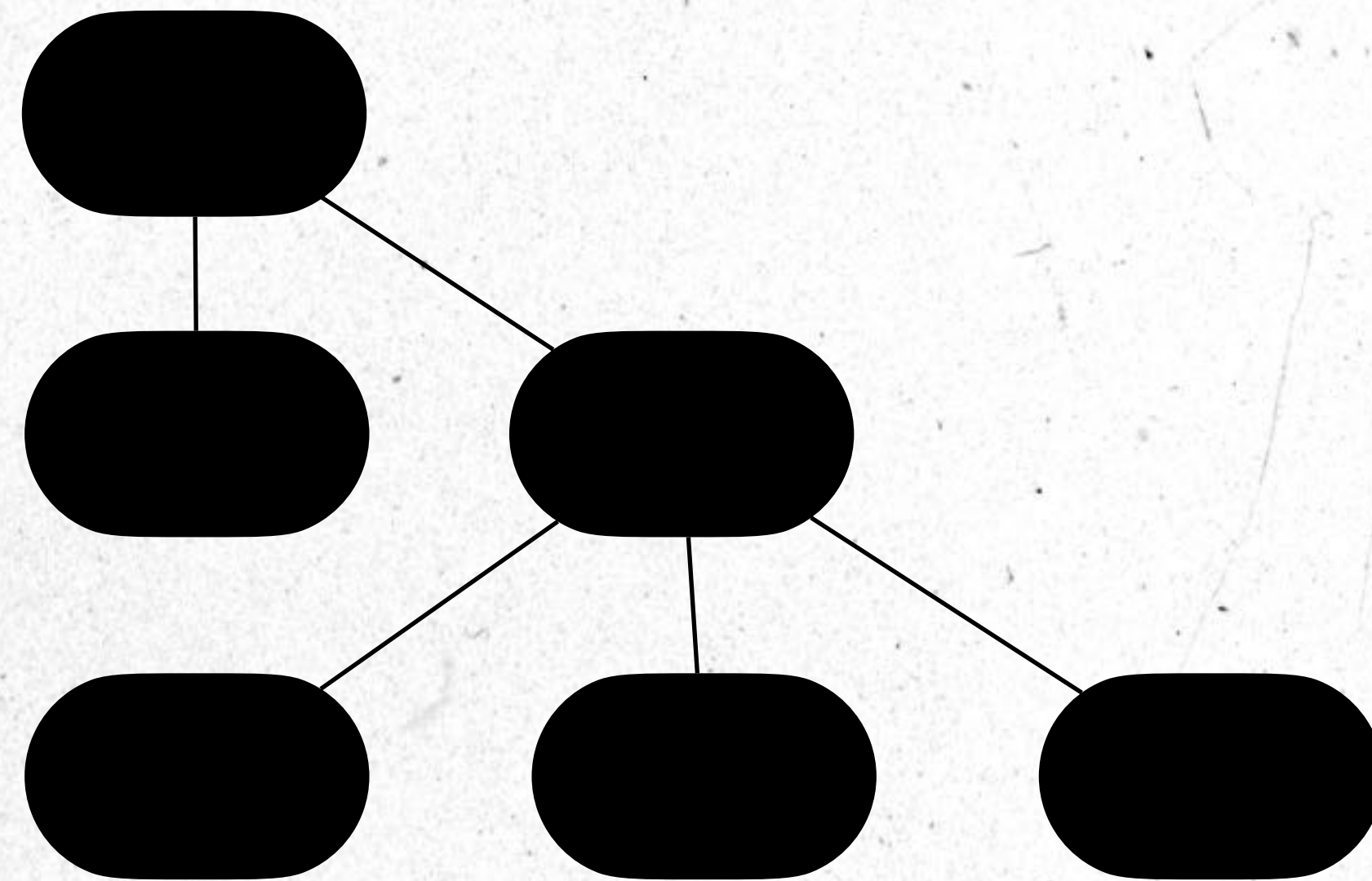






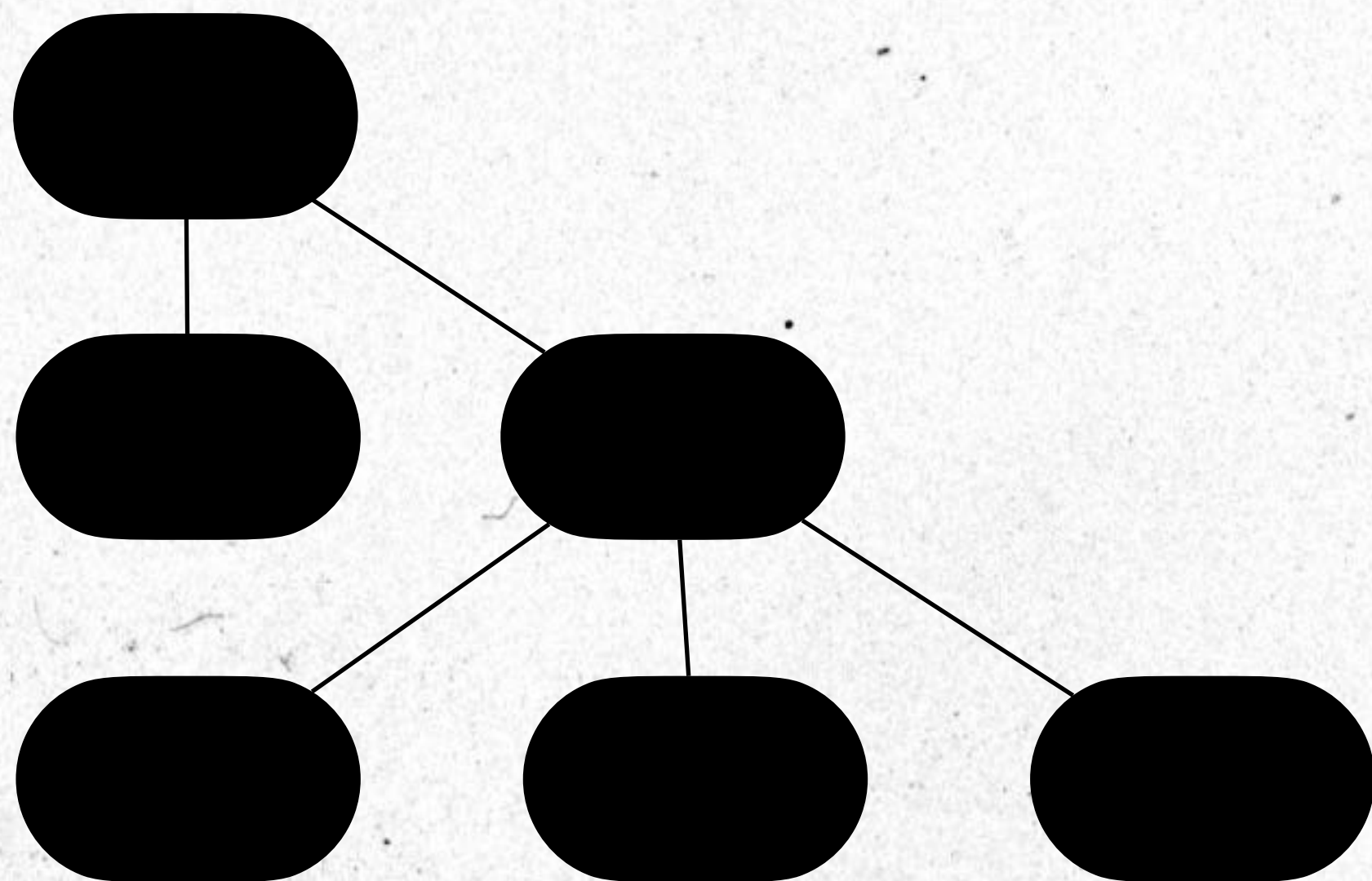
Special Composite Layer

Children layers



Special Composite Layer

Children layers



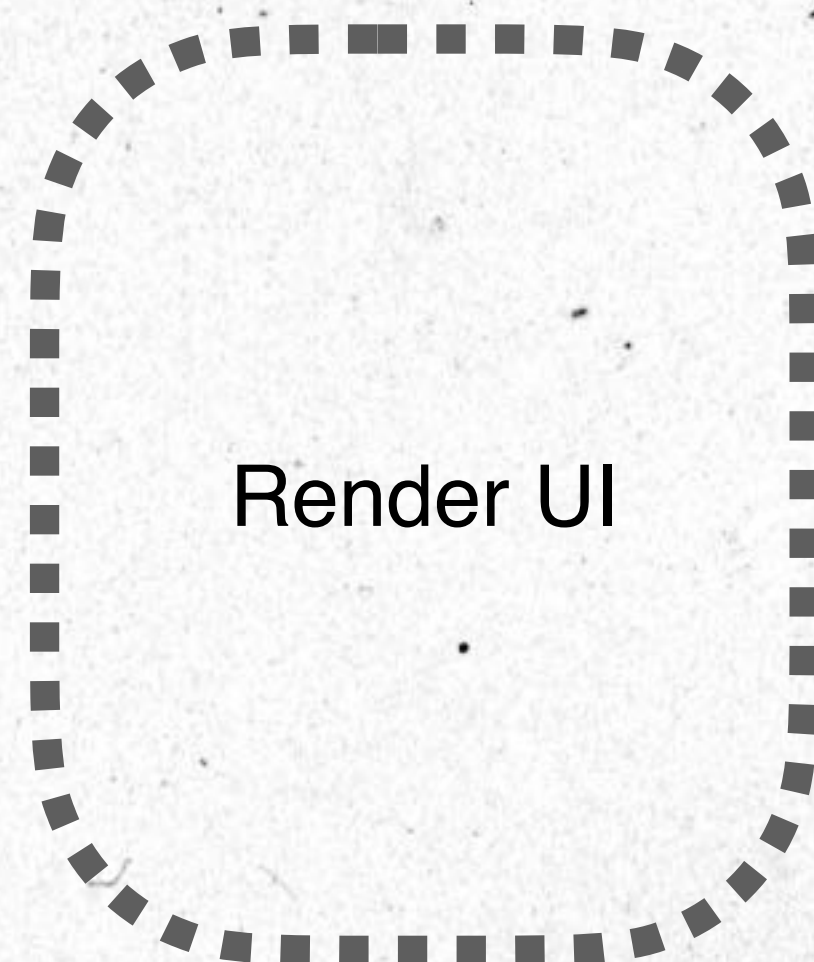
```
final sb = SceneBuilder();
addChildToScene(sb);
Image image = sb.build().toImageSync();

final pictureRecorder = PictureRecorder();
final canvas = Canvas(pictureRecorder);

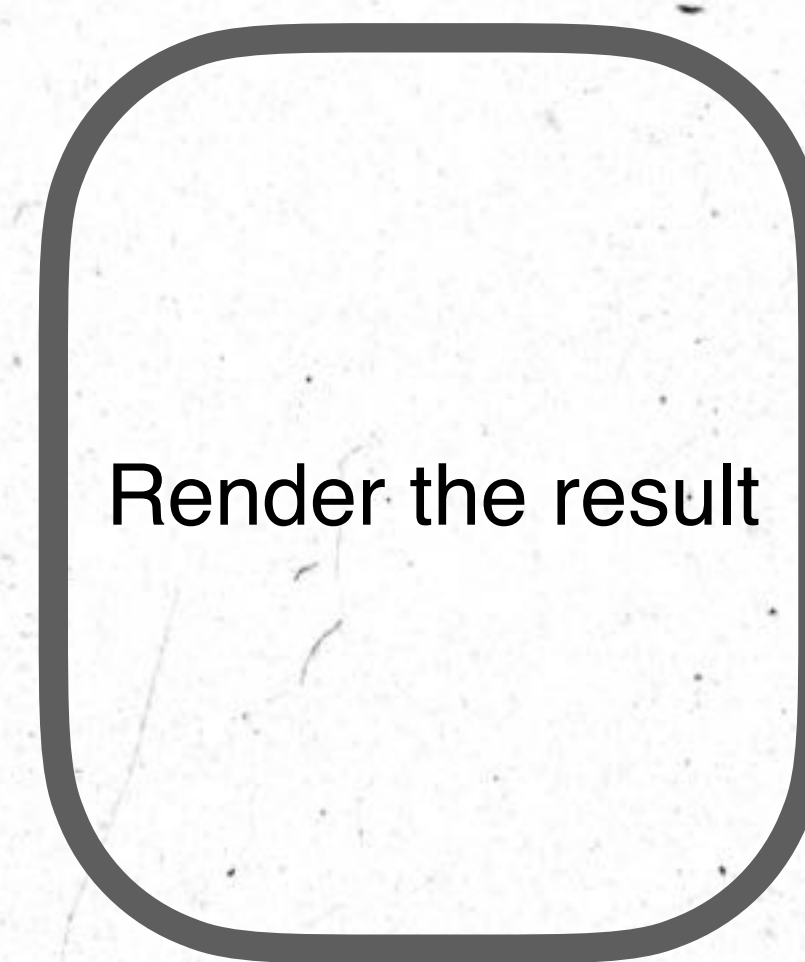
// ..

shader.setImageSampler(0, image);
canvas.drawRect(
    Offset.zero & size,
    Paint()..shader = shader,
);
```

(Not the actual code, this is very simplified)



Process
the result
with shader

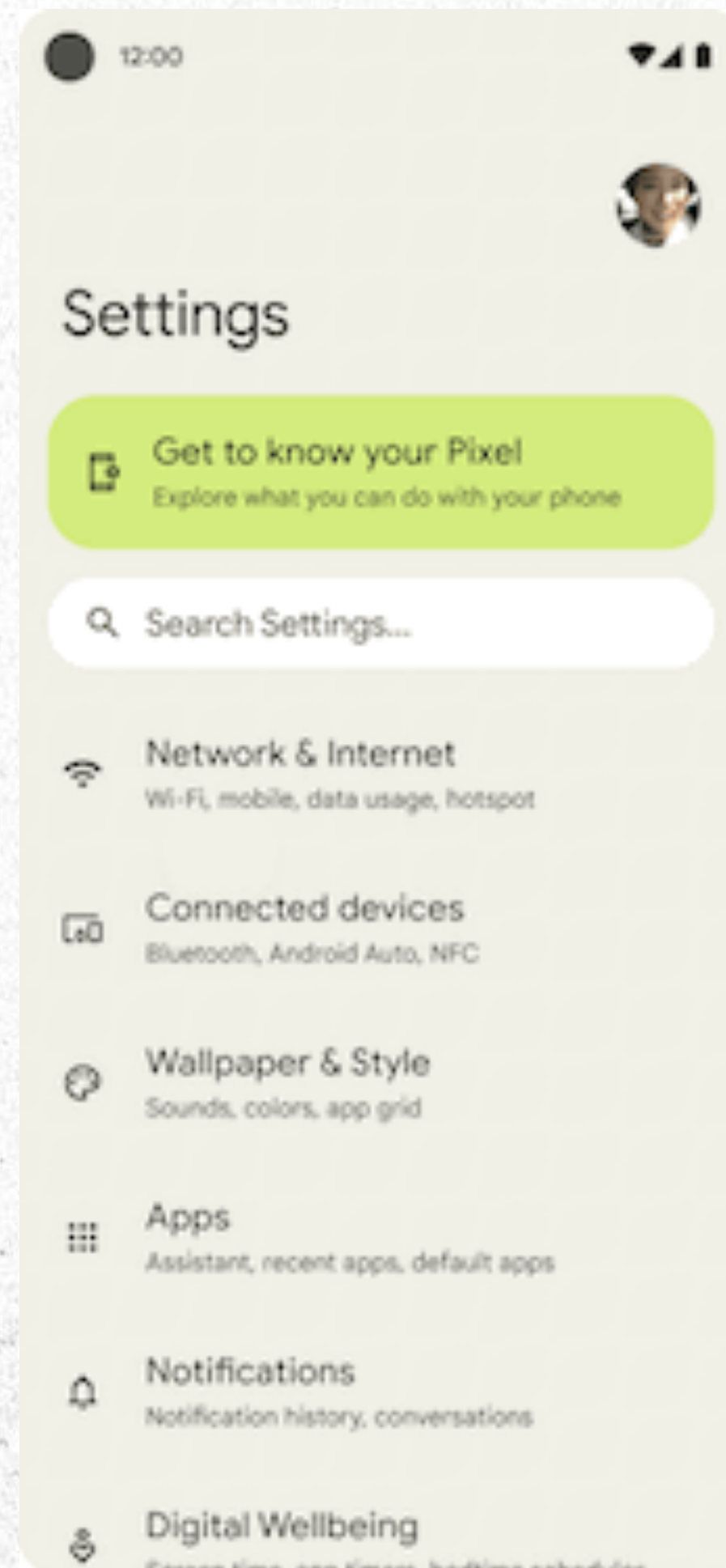


Process
the result
with shader

Render UI

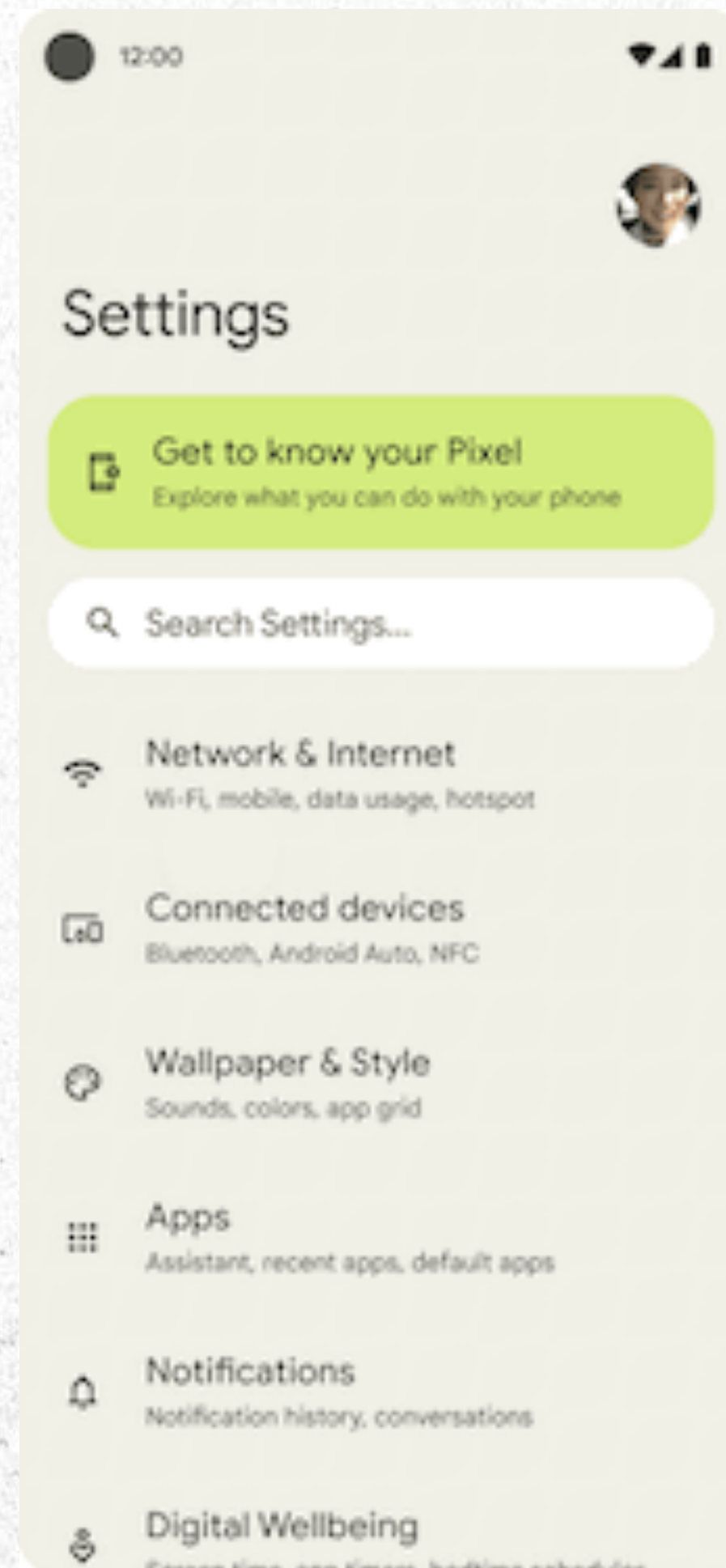
Render the result

This is Post process



Material 3 overscroll

Uses overscroll values to distort scrollable children



Material 3 overscroll

Uses overscroll values to distort scrollable children

3. ImageFilter.shader

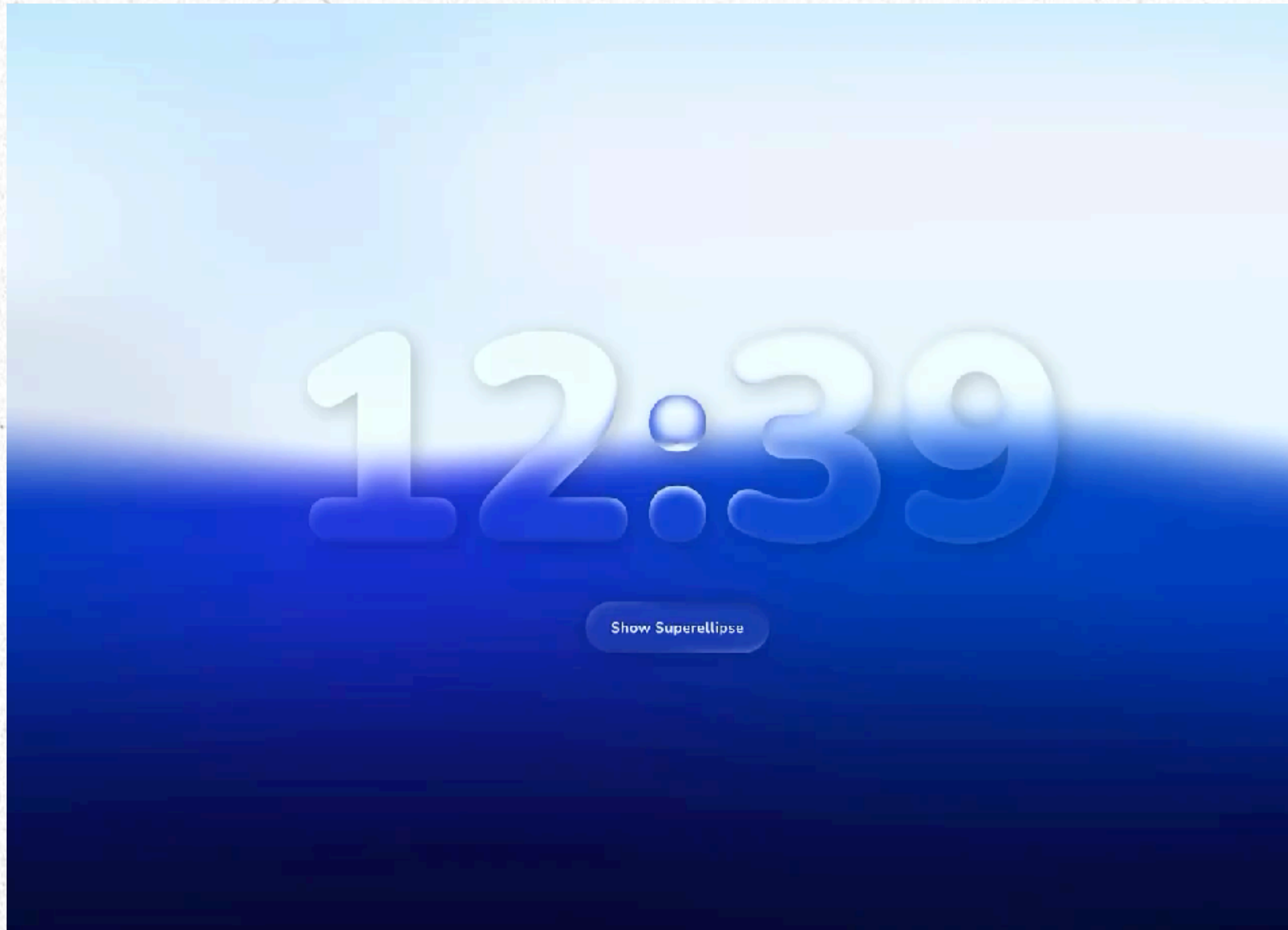

```
ImageFiltered(  
  imageFilter: ImageFilter.shader(  
    shader,  
  ),  
  child: child,  
);
```

```
BackdropFilter(  
  filter: ImageFilter.shader(  
    shader,  
  ),  
  child: child,  
);
```

ImageFiltered

Impeller only

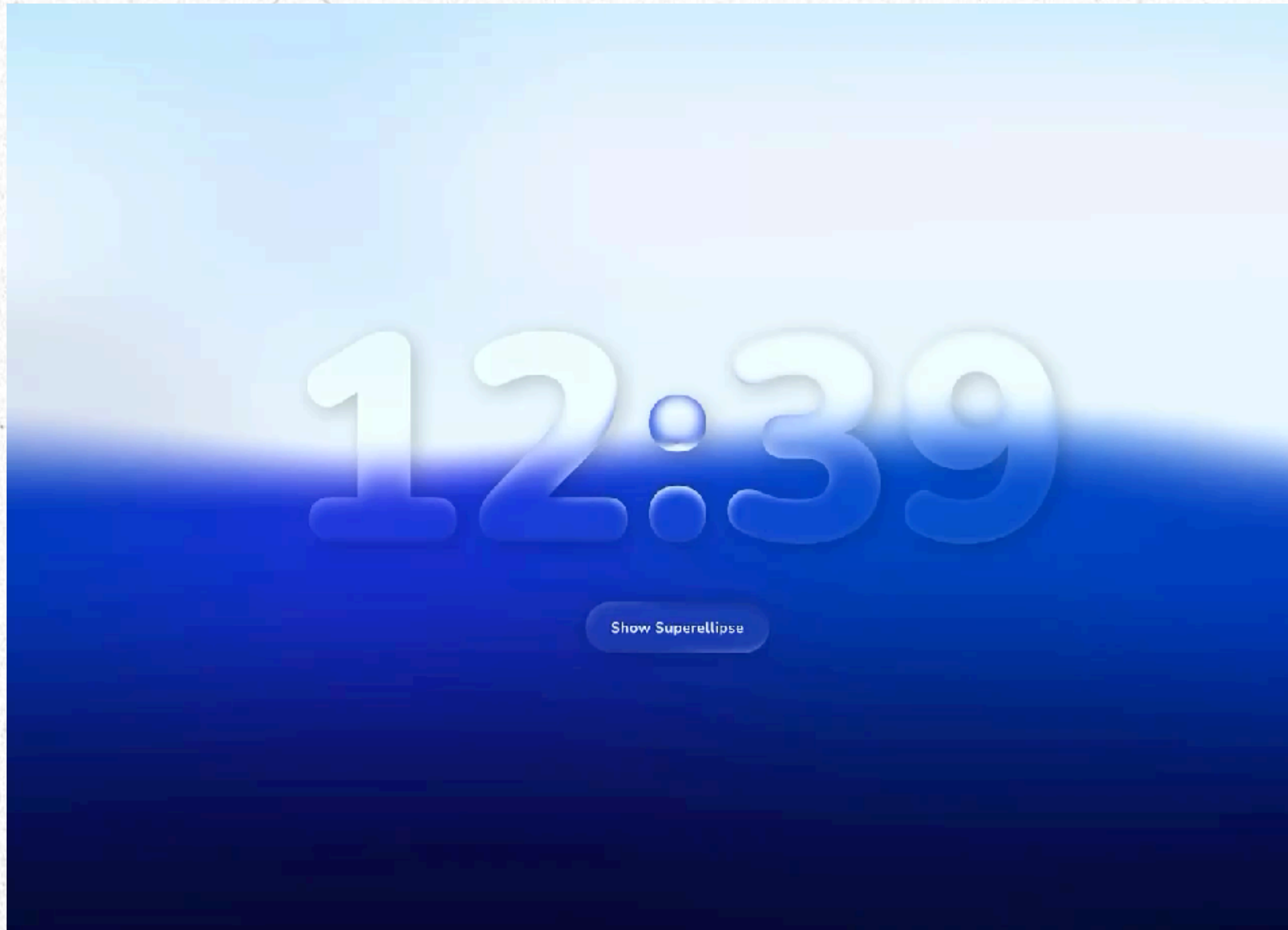




Liquido

Uses both the children and backdrop as textures for the shader.

liquido.renan.gg



Liquido

Uses both the children and backdrop as textures for the shader.

liquido.renan.gg

22:26

22:26

22:26

22:26

22:26

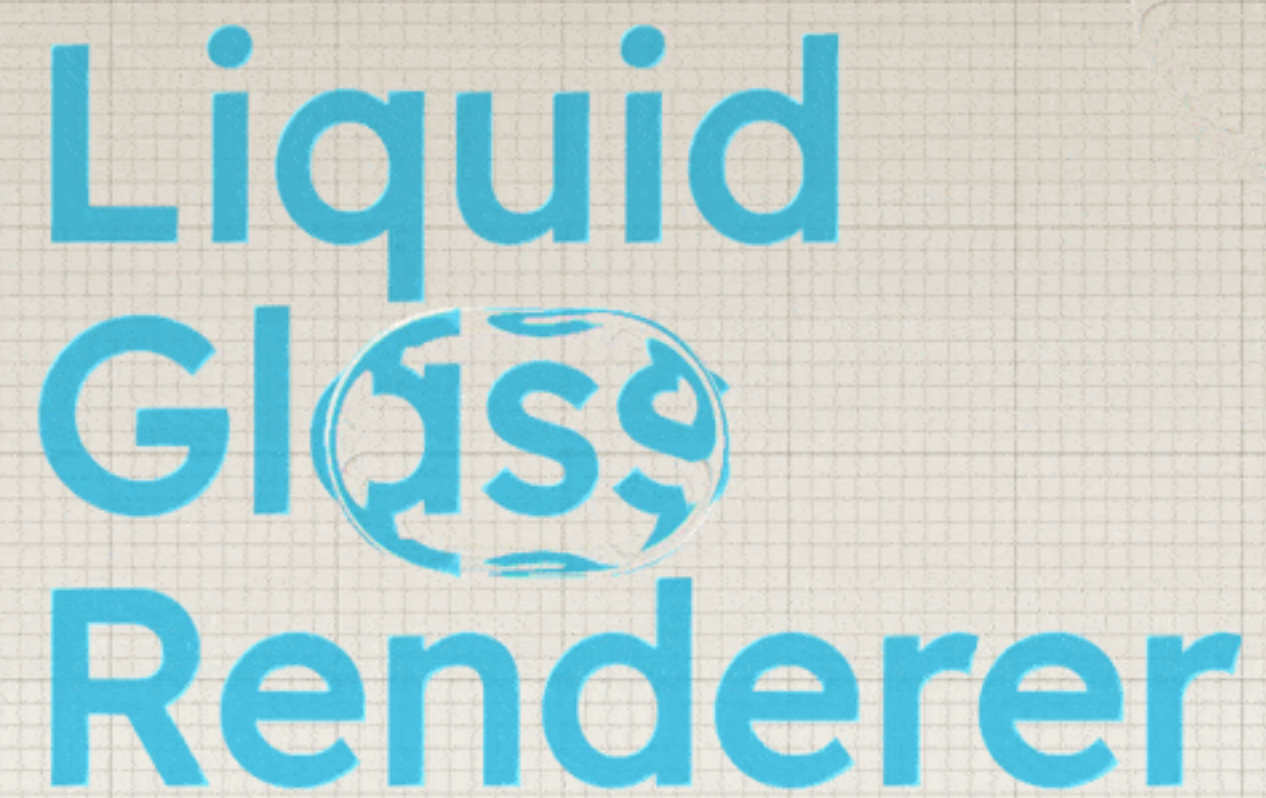
22:26



NAN

@reNotANumber

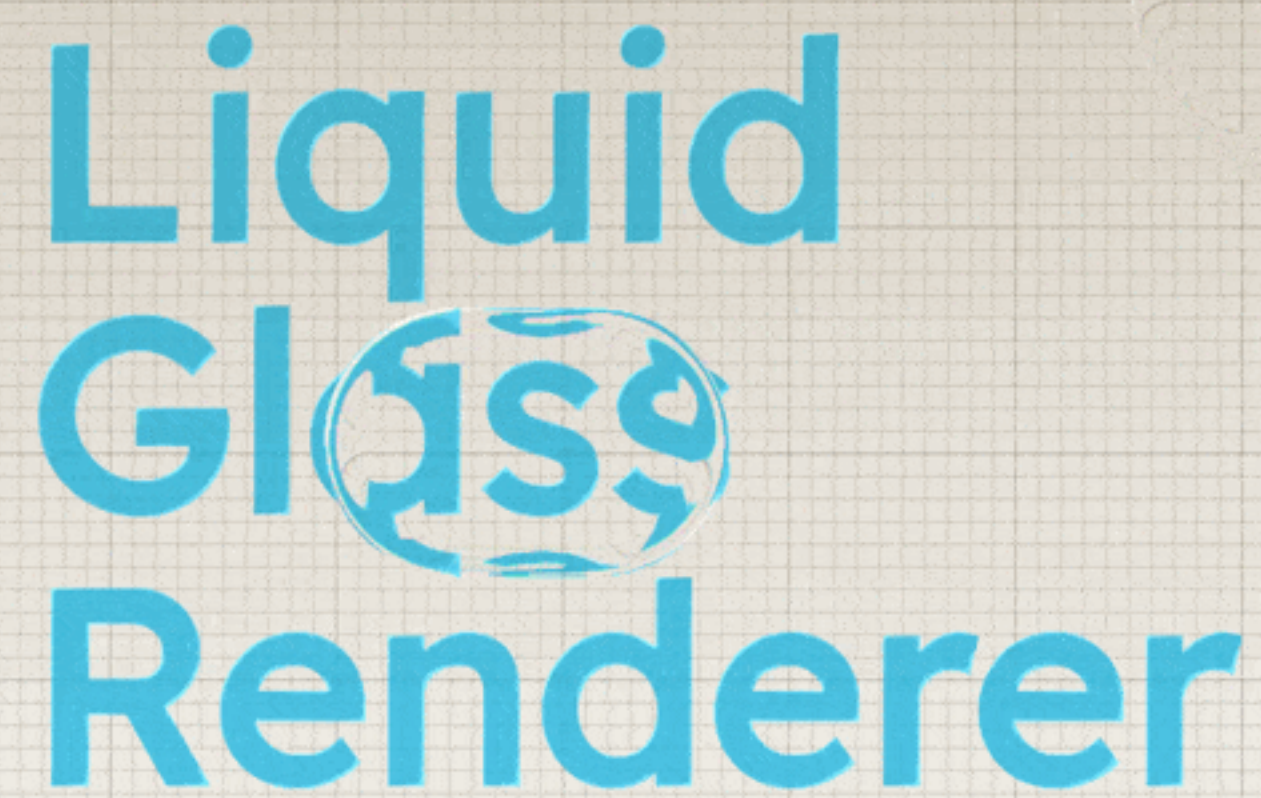
22:26

The logo for 'Liquid Glass Renderer' is displayed in a blue, sans-serif font. The word 'Liquid' is on the top line, 'Glass' is on the second line with a stylized globe icon replacing the letter 'o', and 'Renderer' is on the third line. The background of the logo is a light beige grid pattern.

Liquid
Glass
Renderer

Liquid glass renderer

pub.dev/packages/liquid_glass_renderer

The logo for Liquid Glass Renderer is displayed on a light brown grid background. It features the text "Liquid Glass Renderer" in a blue, sans-serif font. The word "Glass" is partially obscured by a circular icon containing a stylized globe with latitude and longitude lines. Above the text, there are two faint, light blue icons: a cloud on the left and a rounded square containing a stylized 'L' on the right.

Liquid
Glass
Renderer

Liquid glass renderer

pub.dev/packages/liquid_glass_renderer

Flame


```
class MyGame extends FlameGame {
```

```
// ...
```

```
GameWidget(game: MyGame())
```



```
class MyComponent extends PositionComponent {  
    MyComponent() {  
        add(AnotherComponent());  
    }  
  
    @override  
    void update(double dt) {  
        // Update component state  
    }  
  
    @override  
    void render(Canvas canvas) {  
        // Render component  
    }  
}
```


Flutter

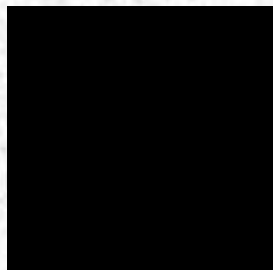
Flame

Fragment Shaders

Flame is a canvas based API

```
class MyComponent extends PositionComponent

    @override
    void render(Canvas canvas) {
        // Render component
    }
}
```



- **Flame Component System**
- **Effects API**
- **Camera API**
- **Collision detection**
- **Raycasting**
- **Sprite and Sprite animations**
- **Particle system**
- **Overlays**
- **Router**
- **Gesture and Multidrag detection**
- **Post-processing**

- **Flame Audio**
- **Forge 2D (Box2d)**
- **Gamepads**
- **Bloc & Riverpod bridges**
- **Flame Tiled**
- **Flame SVG**
- **Flame 3D**
- **Oxygen**
- **Lottie**
- **Flame behaviors**
- **Bonfire**
- **Flame Jenny**
- **Spine**
- **Flame Devtools**

...



Flame's **Post Processing API**

Flame's **Post-processing** API

NAN

postprocessing.renan.gg



@reNotANumber

“The Post Processing API”

Flutter & Friends

3 weeks ago, free to watch on YouTube

“Flutter, Flame & Fragment shaders”

Fluttercon

Today, now... Duh

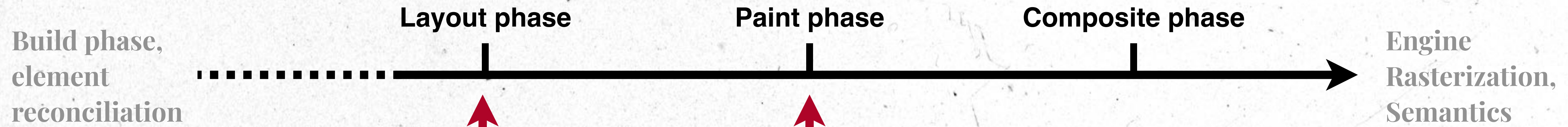
“Post Pocessing UIs”

Flutter Netherlands

October

@reNotANumber







```
void main() {  
    runApp(GameWidget(game: MyGame()));  
}
```

```
class MyGame extends FlameGame {  
    @override  
    Future<void> onLoad() async {  
        world.add(  
            EmberPlayer(),  
        );  
    }  
}
```




```
void main() {  
    runApp(GameWidget(game: MyGame()));  
}
```

```
class MyGame extends FlameGame {  
    @override  
    Future<void> onLoad() async {  
        world.add(  
            EmberPlayer(),  
        );  
    }  
}
```



```
#version 460 core

precision highp float;

#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;
uniform vec2 uPixels;
uniform sampler2D uTexture;

out vec4 fragColor;

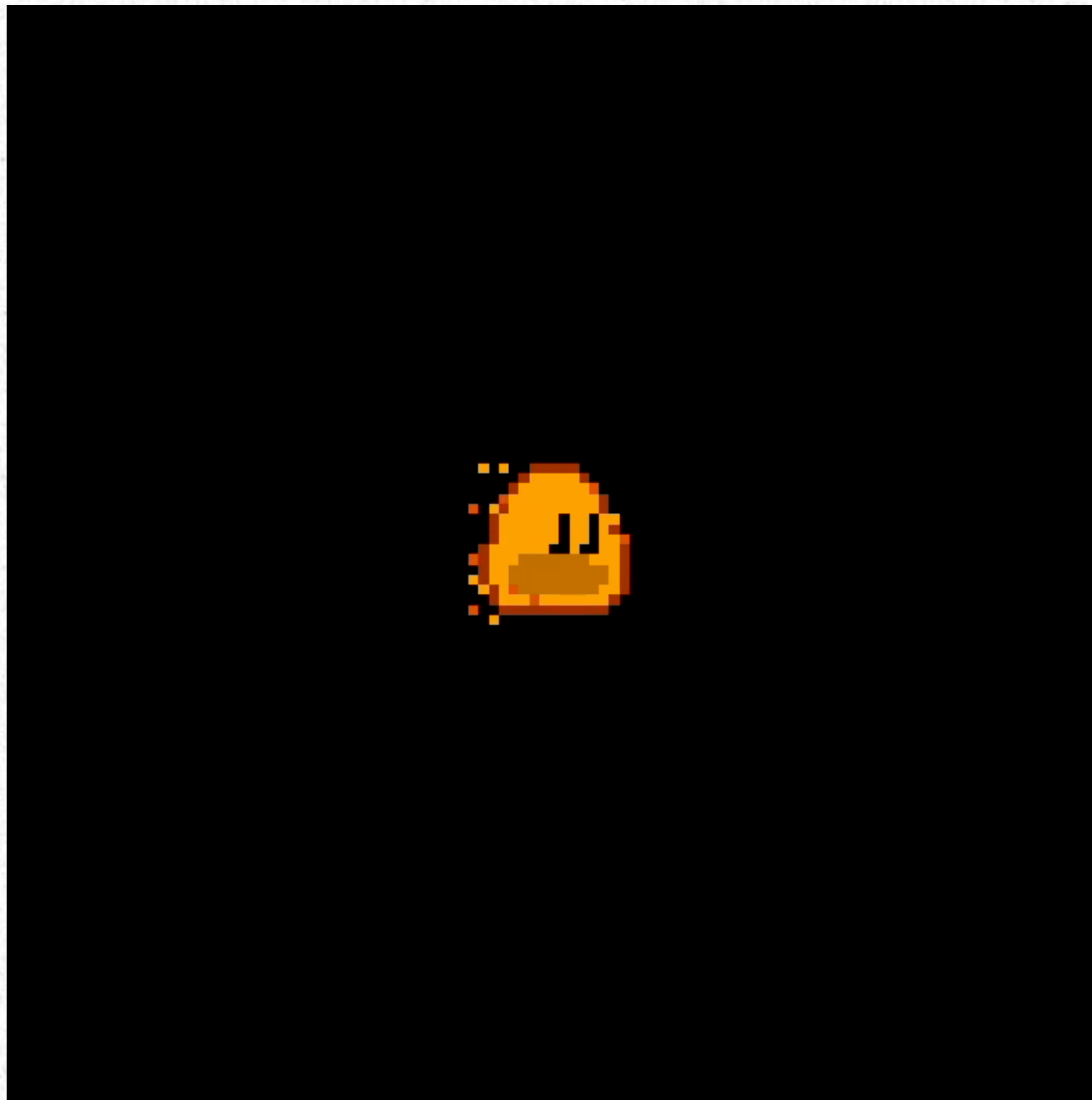
void main() {
    vec2 uv = FlutterFragCoord().xy / uSize;
    vec2 puv = round(uv * uPixels) / uPixels;
    fragColor = texture(uTexture, puv);
}
```




```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```

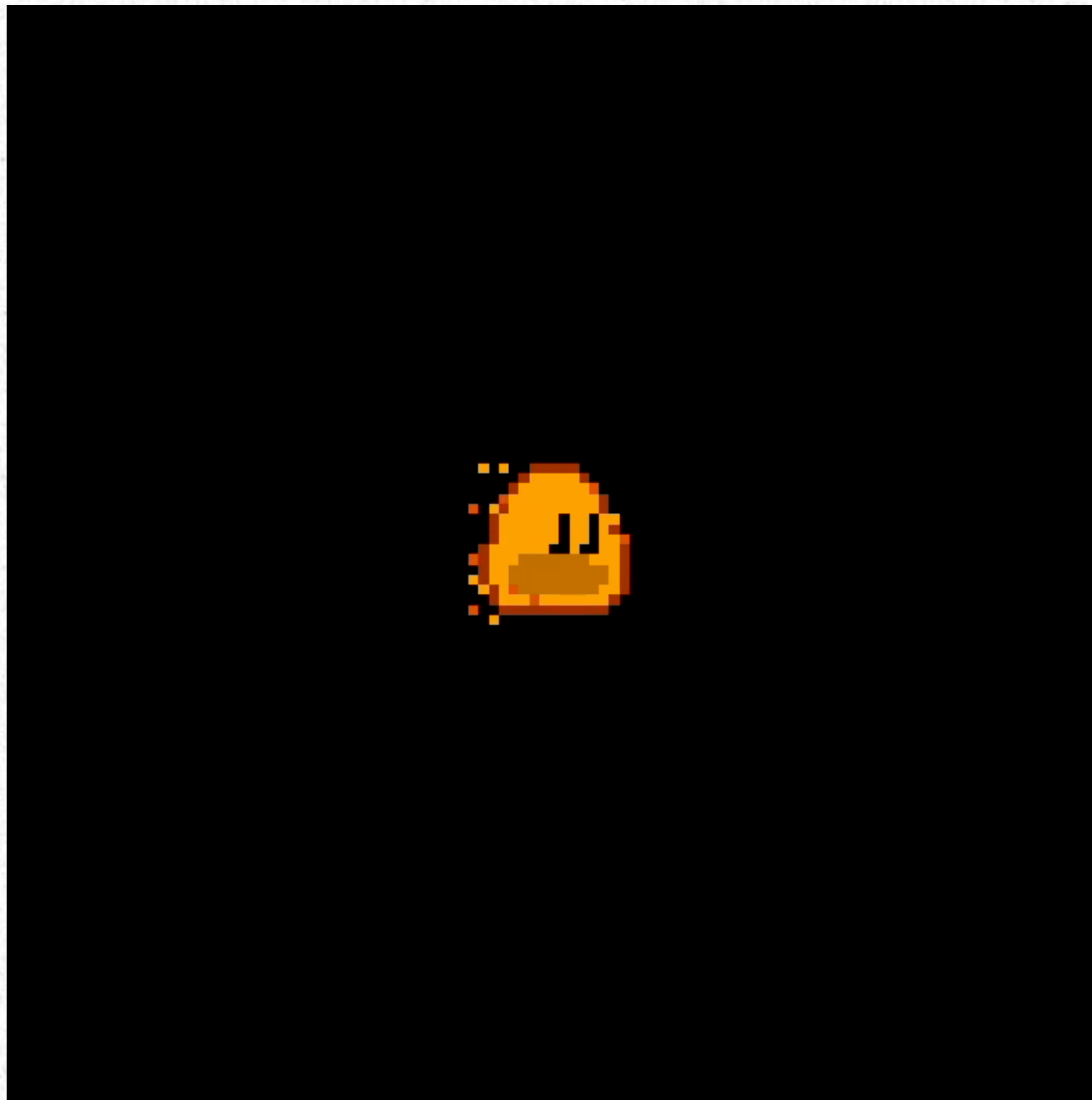



```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```

```
import 'package:flame/post_process.dart';
```

```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```

```
import 'package:flame/post_process.dart';
```

```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```



```
import 'package:flame/post_process.dart';
```

```
class PixelationPostProcess extends PostProcess {  
    ...  
}
```



```
import 'package:flame/post_process.dart';
```

```
class PixelationPostProcess extends PostProcess {  
    ...  
}
```




```
@override
Future<void> onLoad() async {
    _fragmentShader = (await ui.FragmentProgram.fromAsset(
        'shaders/my_shader.frag',
    )).fragmentShader();
}

late final ui.FragmentShader _fragmentShader;
late final myPaint = Paint()..shader = _fragmentShader;

@override
void postProcess(Vector2 size, Canvas canvas) {

}
```




```
@override
Future<void> onLoad() async {
    _fragmentShader = (await ui.FragmentProgram.fromAsset(
        'shaders/my_shader.frag',
    )).fragmentShader();
}

late final ui.FragmentShader _fragmentShader;
late final myPaint = Paint()..shader = _fragmentShader;

@override
void postProcess(Vector2 size, Canvas canvas) {

}
```




```
final ui.Image preRenderedSubtree = rasterizeSubtree();

_fragmentShader.setFloatUniforms((value) {
    value
        ..setVector(size)
        ..setVector(Vector2(4, 4));
});

_fragmentShader.setImageSampler(0, preRenderedSubtree);

canvas
    ..save()
    ..drawRect(ui.Offset.zero & size.toSize(), myPaint)
    ..restore();
```

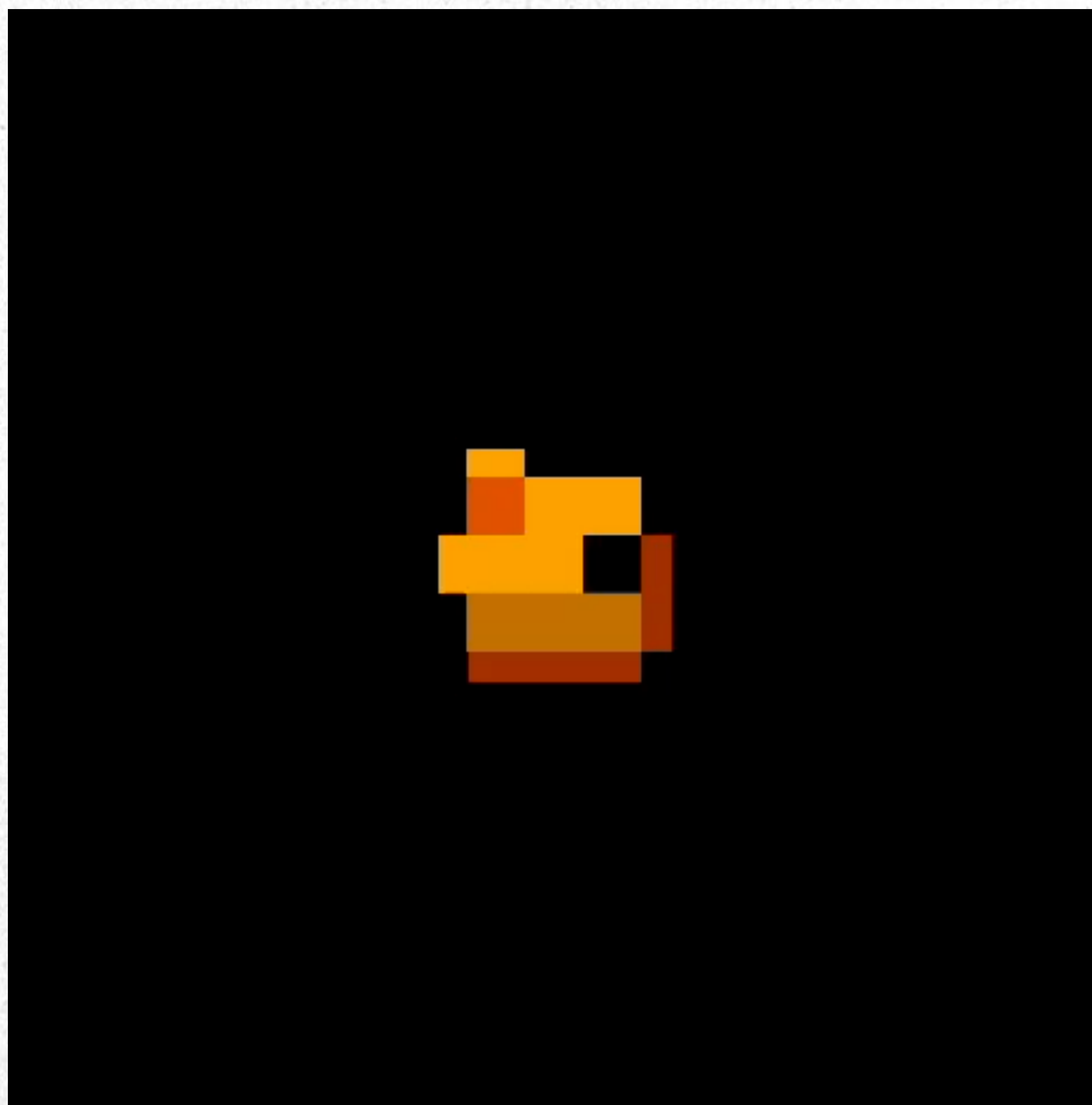



```
final ui.Image preRenderedSubtree = rasterizeSubtree();

_fragmentShader.setFloatUniforms((value) {
    value
        ..setVector(size)
        ..setVector(Vector2(4, 4));
});

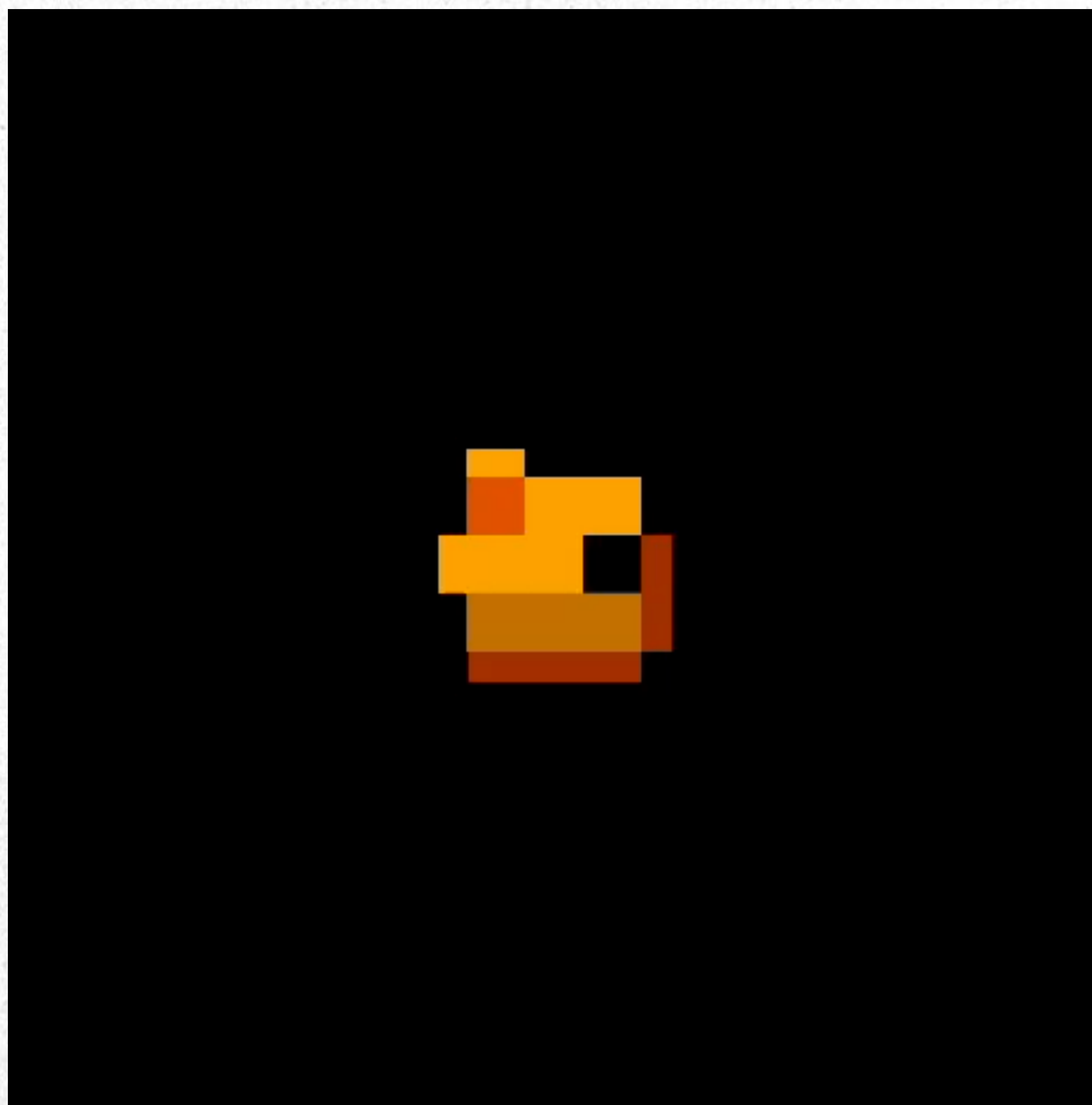
_fragmentShader.setImageSampler(0, preRenderedSubtree);

canvas
    ..save()
    ..drawRect(ui.Offset.zero & size.toSize(), myPaint)
    ..restore();
```

NAN

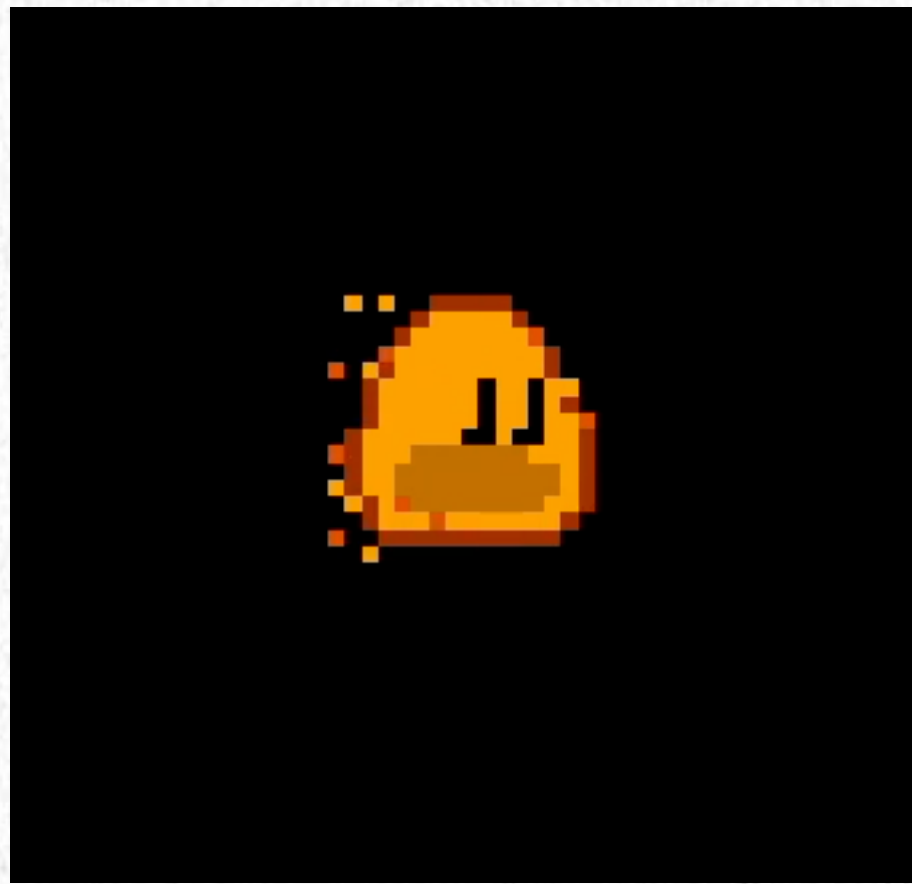
@reNotANumber



NAN

@reNotANumber

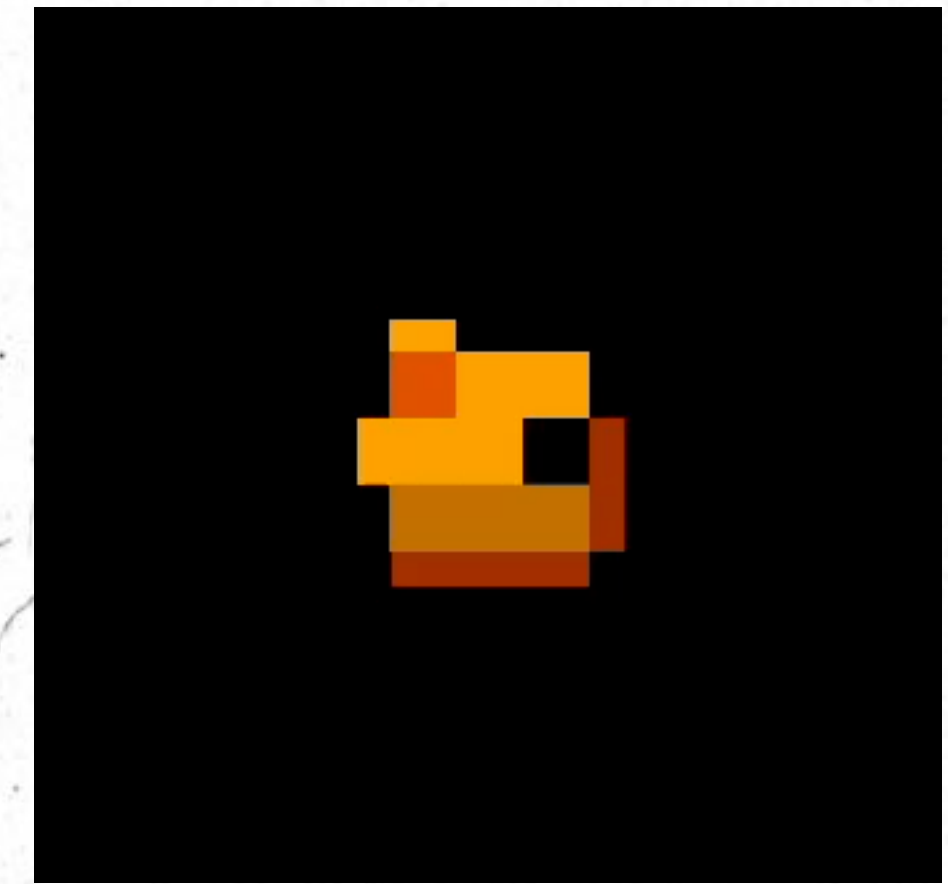
Component renders



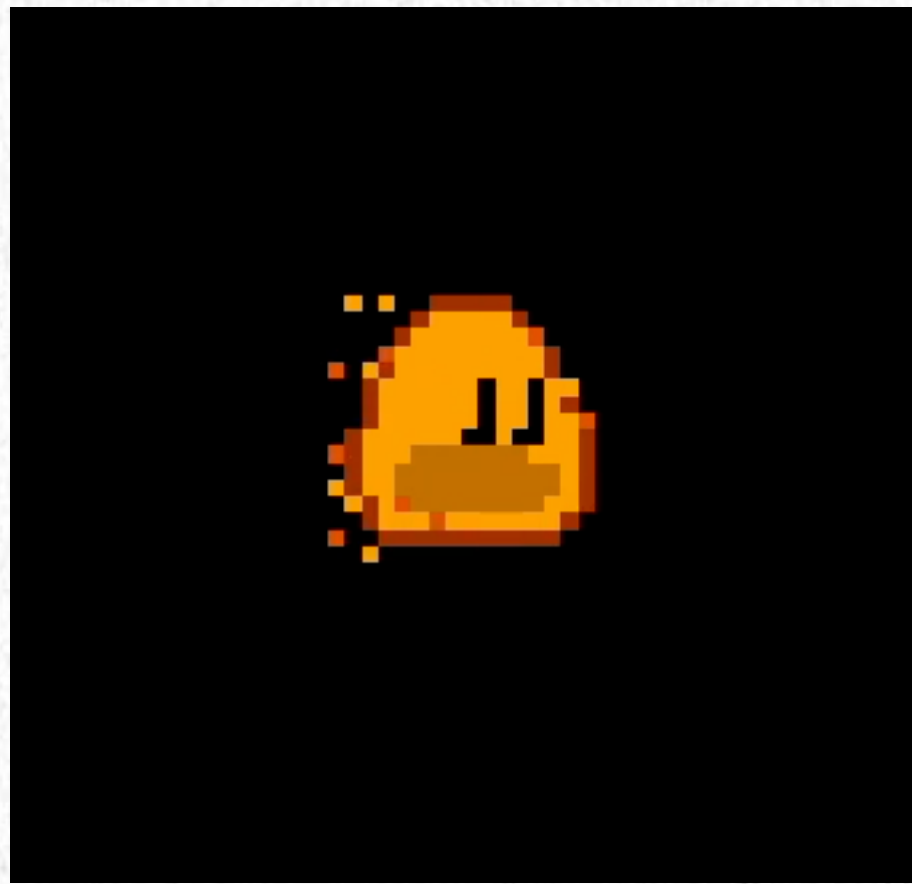
Post-process
sends image to
shader



Shader renders



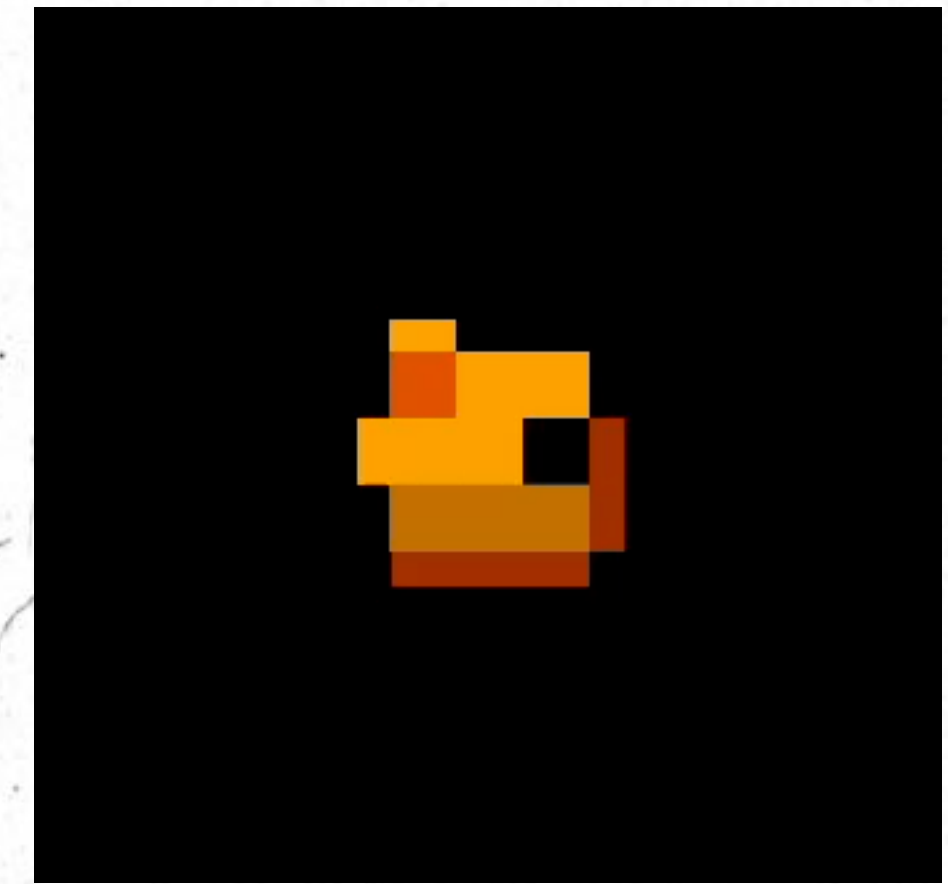
Component renders



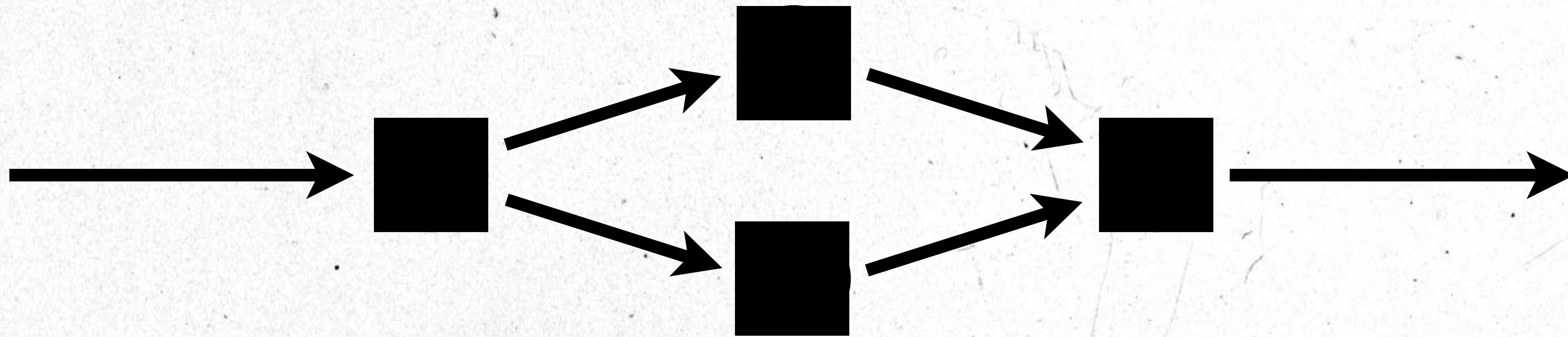
Post-process
sends image to
shader



Shader renders



Pipeline of post processes



⚠ Rasterizing is expensive, dont abuse

github.com/renancaraujo/turi

“Observable Flutter #27:
Flame + Shaders”

examples.flame-engine.org

NAN



github.com/renancaraujo/turi

“Observable Flutter #27:
Flame + Shaders”

examples.flame-engine.org

NAN





NAN

@reNotANumber



NAN

@reNotANumber

I did a thing

NAN

@reNotANumber

@renotanumber

NAN

@reNotANumber

NAN

renan.gg